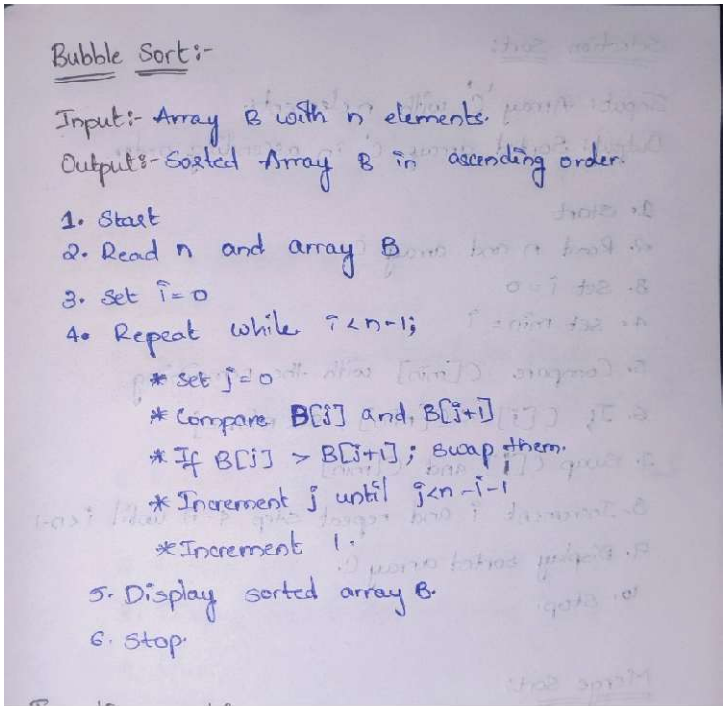
PRACTICAL: 01

Aim: To implement and find the time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.

A) Bubble Sort:

Algorithm:

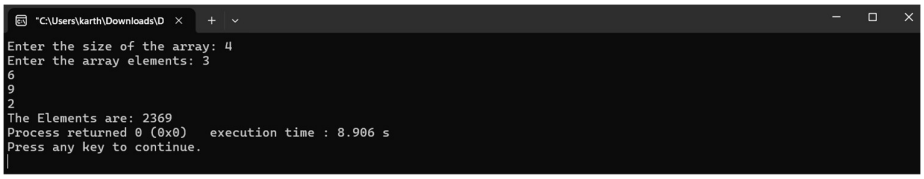


Program / Code:

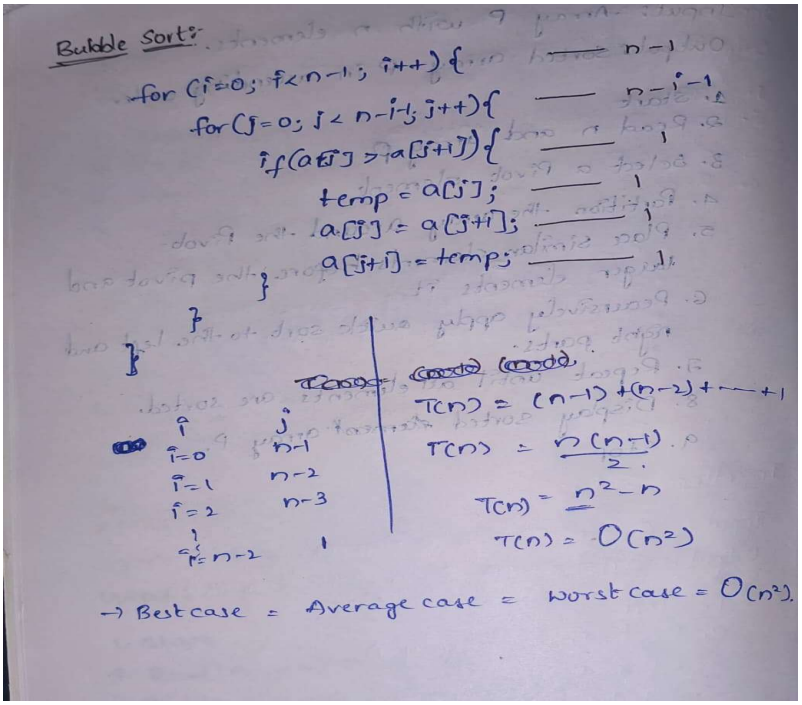
```
#include<iostream>
using namespace std;
int main()
{
   int a[60], size, i, j, temp;
   cout<<"Enter the size of the array: ";
   cin>>size;
   cout<<"Enter the array elements: ";
   for(i=0; i<size; i++){
```

```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
    cout<<"The Elements are: ";
    for(i=0; i<size; i++){
        cout<<a[i];
    }
    return 0;
}
```

Output:

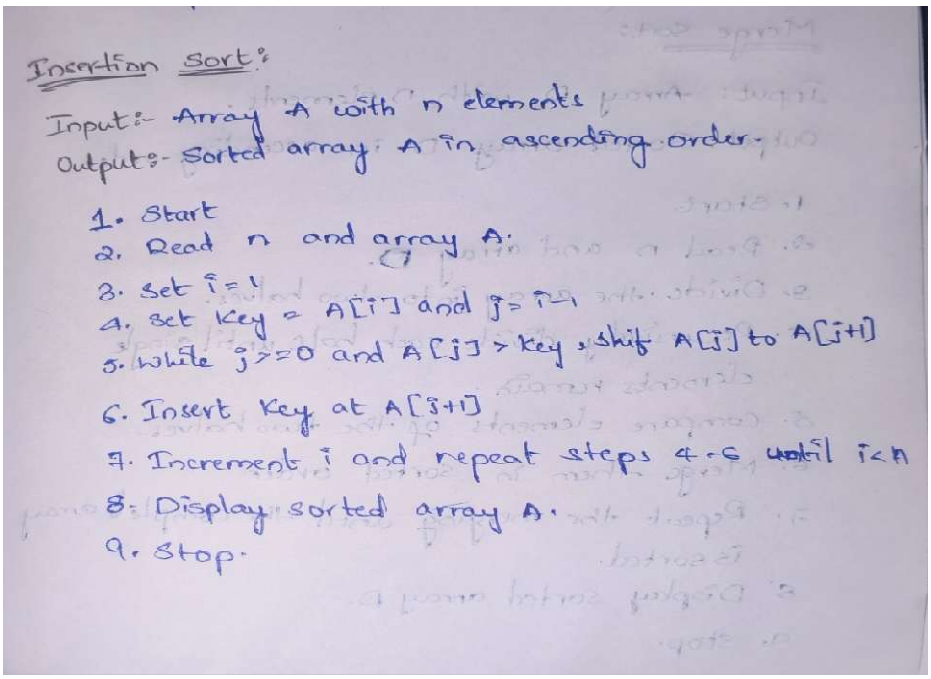


Time Complexity:



B) Insertion Sort:

Algorithm:

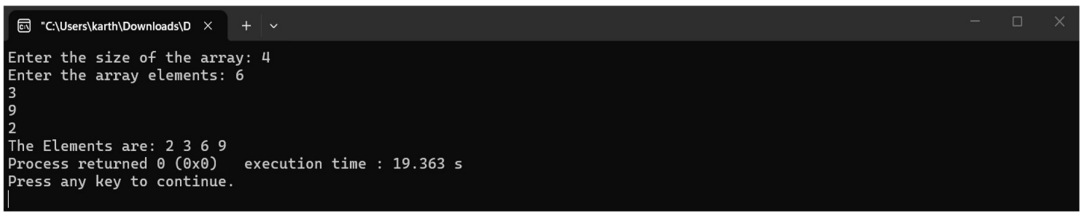


Program / Code:

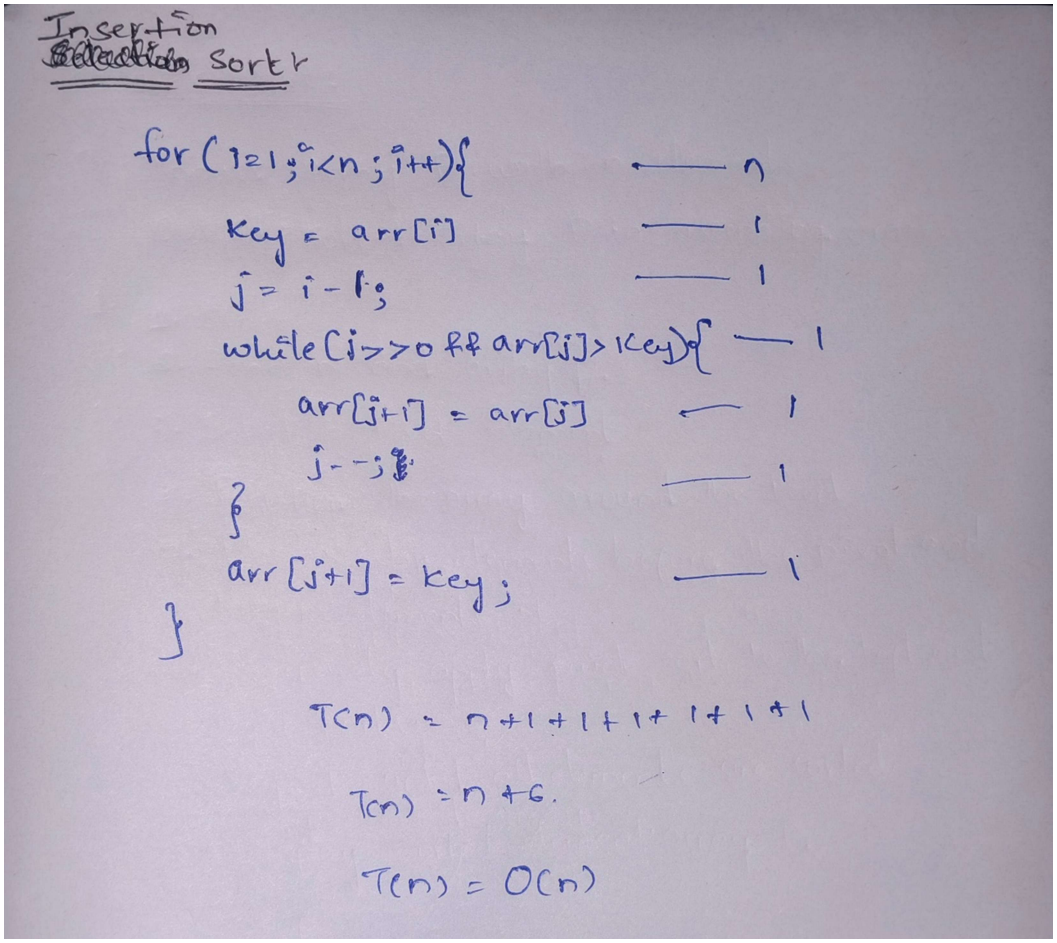
```
#include<iostream>
using namespace std;
int main(){
   int a[60], size, i, j, min, temp;
   cout<<"Enter the size of the array: ";
   cin>>size;
   cout<<"Enter the array elements: ";
   for(i=0; i<size; i++){
      cin>>a[i];
   }
   for(i=0; i<size-1; i++){
      min = i;
      for(j=i+1; j<size; j++){
         if(a[j] < a[min]){
            min = j;
         }
      }
      if(min != i){
         temp = a[i];
         a[i] = a[min];
         a[min] = temp;
      }
   }
   cout<<"The Elements are: ";
   for(i=0; i<size; i++){
      cout<<a[i]<<" ";
```

```
    }
    return 0;
}
```

Output:

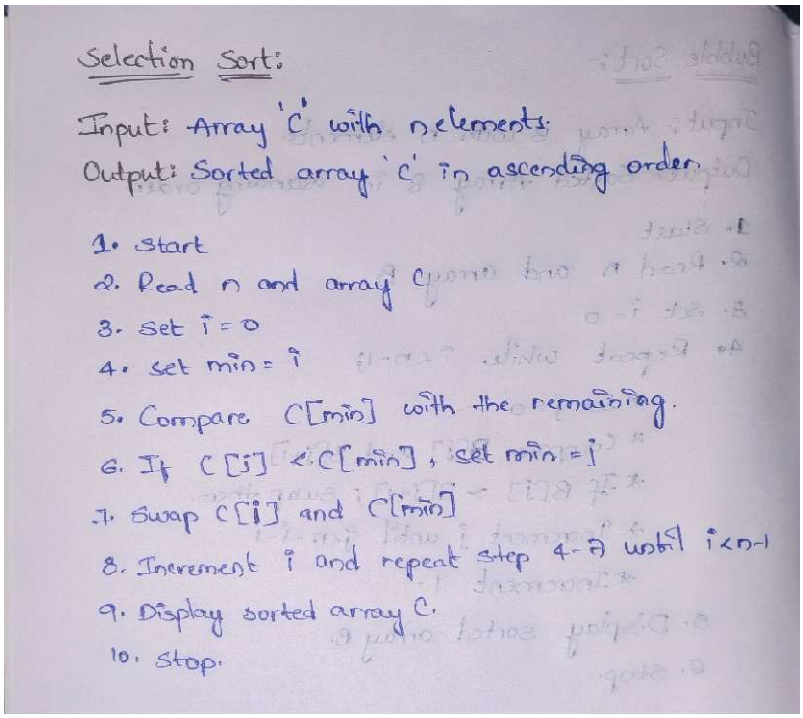


Time Complexity:



C) Selection Sort:

Algorithm:

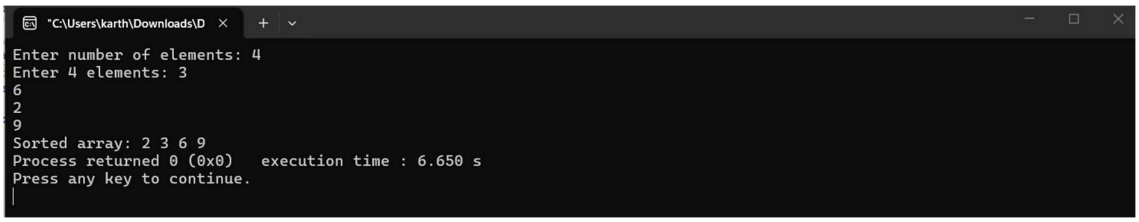


Program / Code:

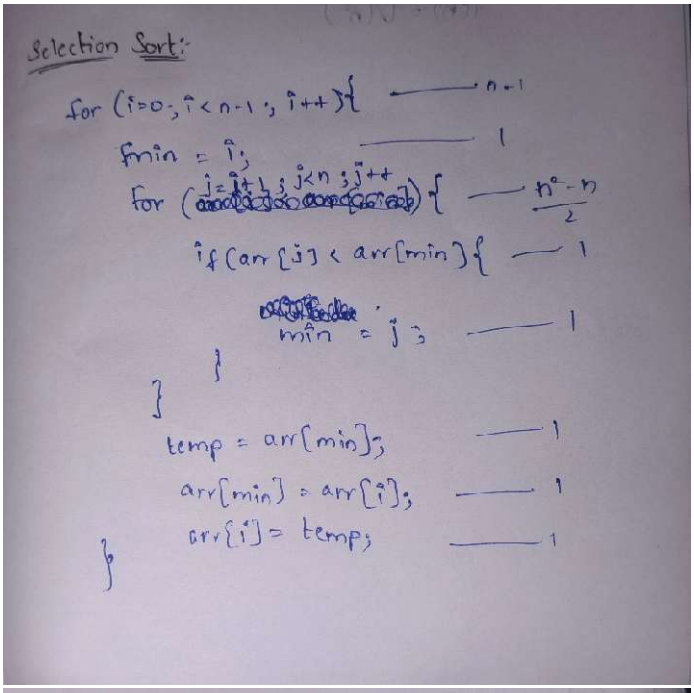
```
#include <iostream>
using namespace std;
void selectionSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
   for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
           minIndex = j;
        }
   }
   int temp = arr[minIndex];
```

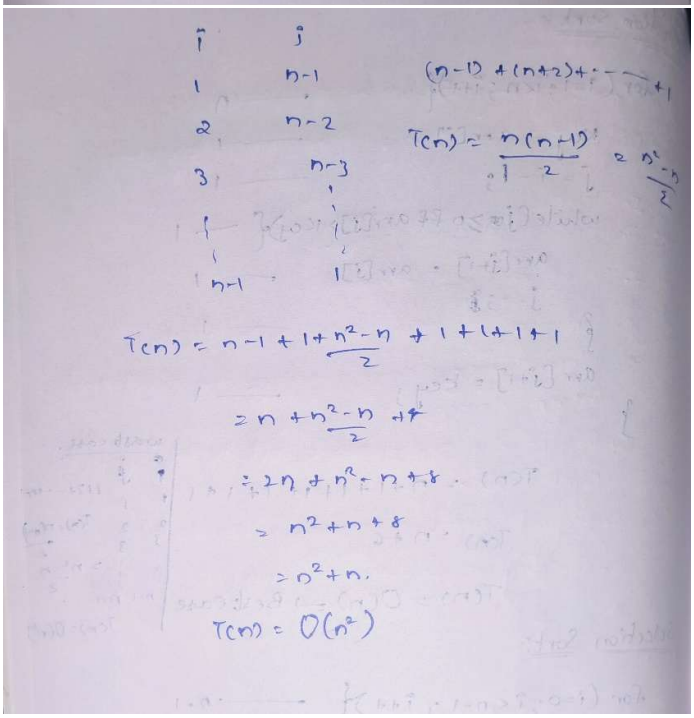
```
    arr[minIndex] = arr[i];
    arr[i] = temp;
    }
}
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    selectionSort(arr, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++){
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:



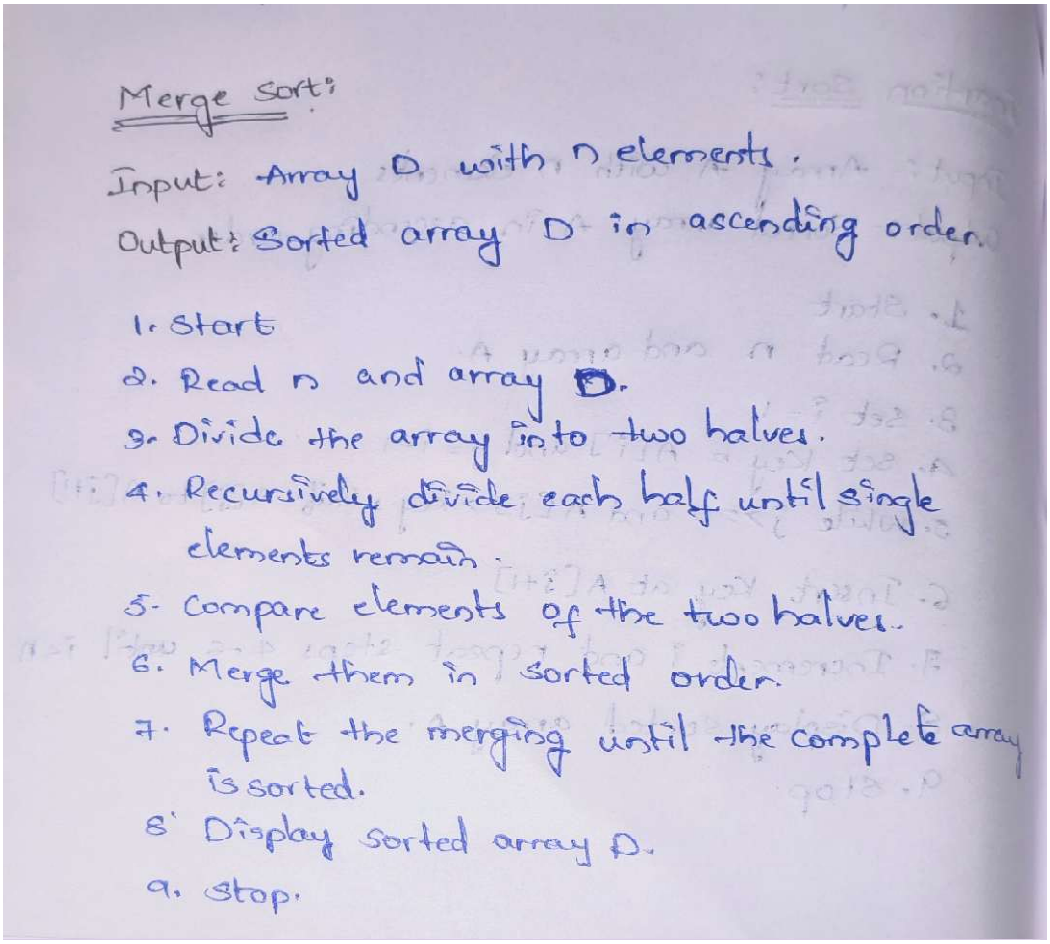
Time complexity:





D) Merge Sort:

Algorithm:



Program / Code:

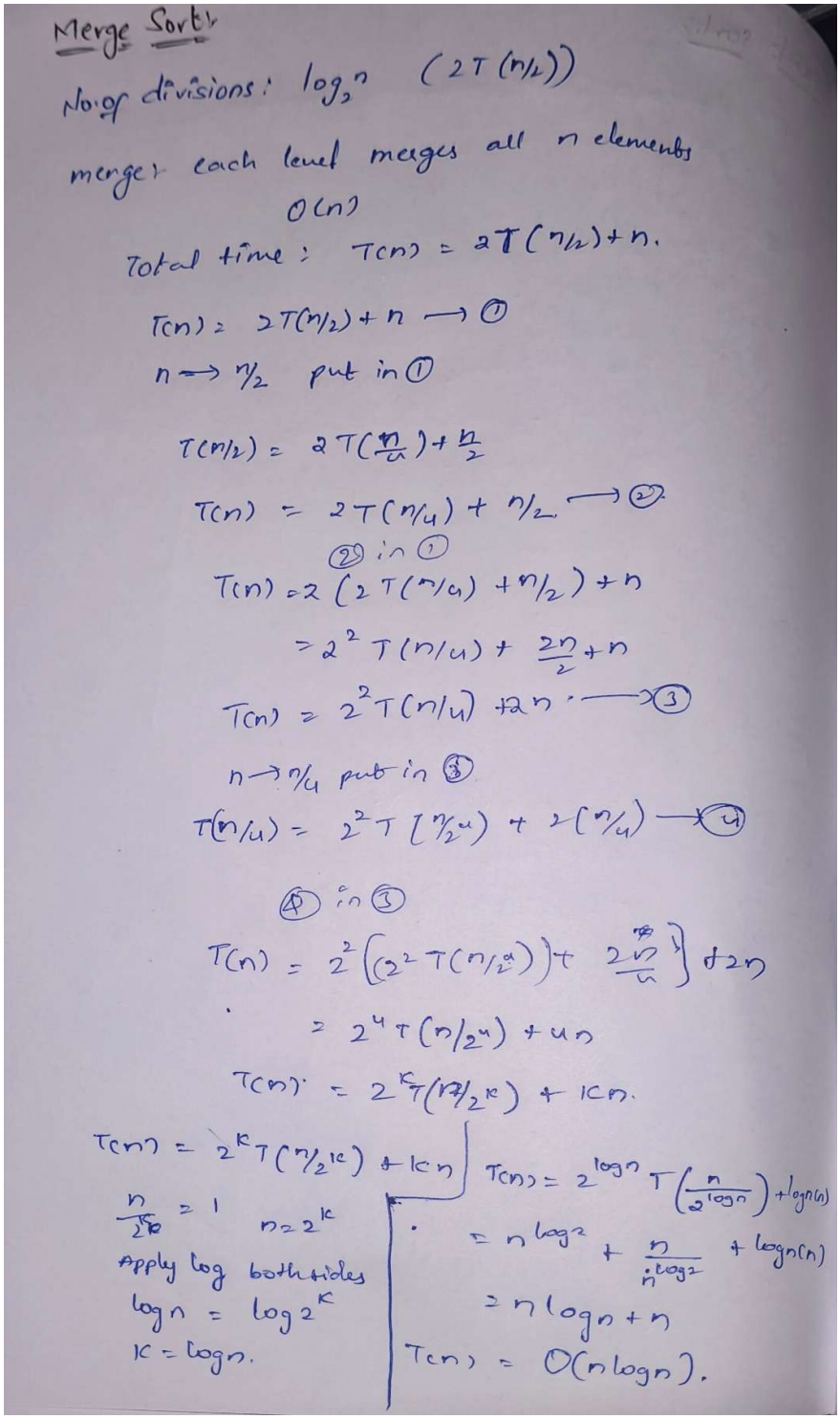
```
#include <iostream>

using namespace std;

void merge(int arr[], int left, int mid, int right) {

    int n1 = mid - left + 1;

    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) {

        L[i] = arr[left + i];
```

```
    }
    for (int j = 0; j < n2; j++) {
        R[j] = arr[mid + 1 + j];
    }
    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
```

```
    }
}
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    mergeSort(arr, 0, n - 1);
    cout << "Sorted array using Merge Sort: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

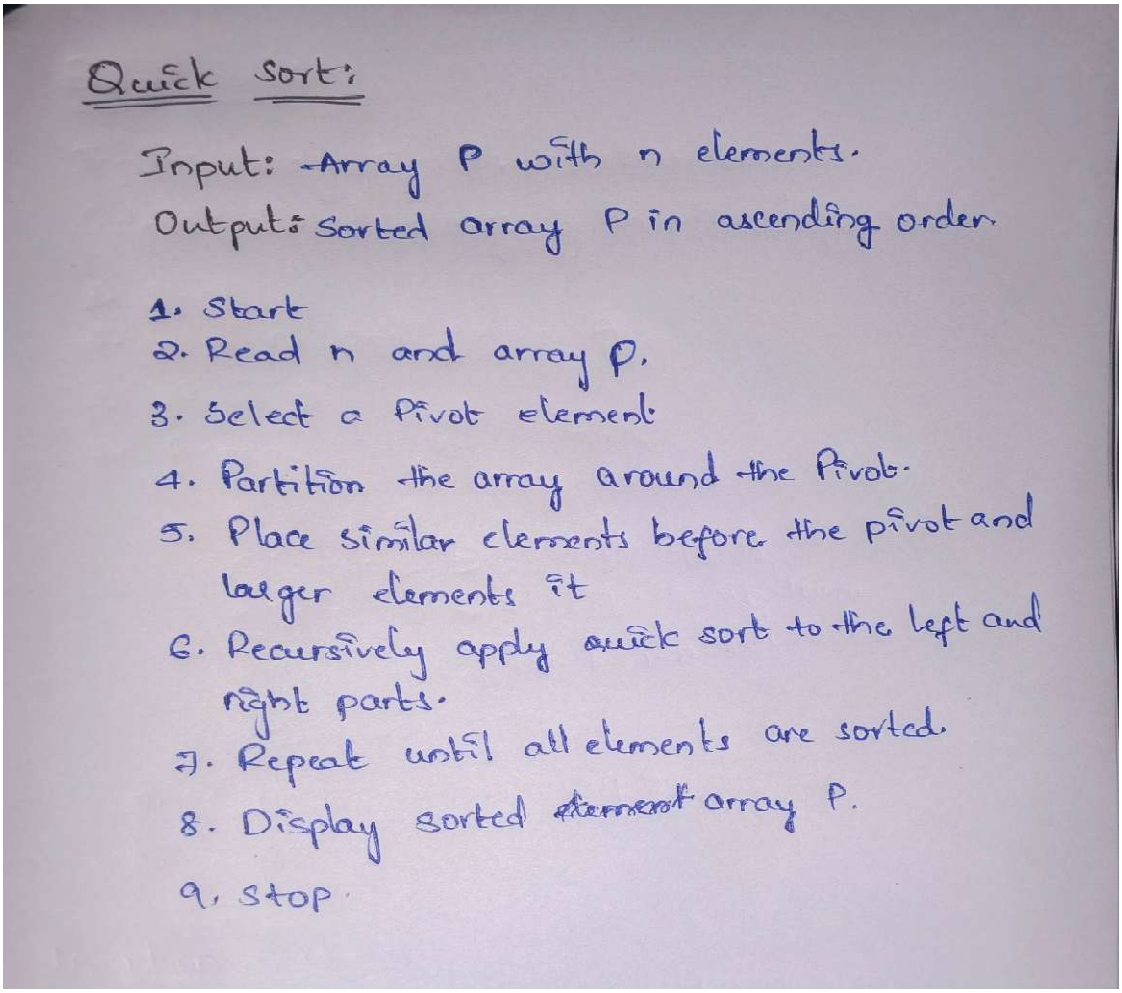
Output:

Time Complexity:



E) Quick Sort:

Algorithm:



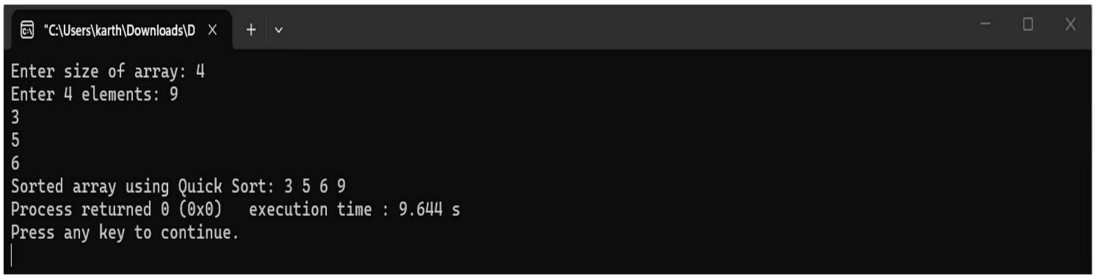
Program / Code:

```
#include <iostream>
using namespace std;
int partition(int arr[], int lb, int ub) {
    int pivot = arr[lb];
    int start = lb;
```

```
    int end = ub;
    while (start < end) {
        while (arr[start] <= pivot) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
        if (start < end) {
            swap(arr[start], arr[end]);
        }
    }
    swap(arr[lb], arr[end]);
    return end;
}
void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1);
        quickSort(arr, loc + 1, ub);
    }
}
int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
```

```
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    quickSort(arr, 0, n - 1);
    cout << "Sorted array using Quick Sort: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:



Time Complexity: